EN4554 Assignment 2: Convolutional Neural Networks

Index number: 220343B
Name: Rivindu Vinsara Kumarage
GitHub repo: https://github.com/rivindu02/EN4554-Assignment-2-on-CNNs
Date: October 2026
Dataset: CIFAR-10

Step 0: Setup

This cell imports all required libraries, fixes random seeds for reproducibility, loads CIFAR-10, splits 45k/5k train/validation, and normalises using **training-set statistics only**.
The whole dataset is kept as in-memory tensors rather than using a DataLoader, which (a) speeds up training for a dataset this small and (b) makes writing custom augmentations in Task 3 straightforward.

In [4]:
```
# ── imports ──────────────────────────────────────────────────────────
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import random
import time
from torchvision import datasets

print(f"PyTorch  : {torch.__version__}")
print(f"CUDA     : {torch.cuda.is_available()}")
```

```
PyTorch  : 2.11.0+cu130
CUDA     : True
```

In [5]:
```
# ── reproducibility ──────────────────────────────────────────────────
def seed_everything(s: int = 0) -> None:
    """Fix seeds across Python, NumPy, and PyTorch (CPU + GPU)."""
    random.seed(s)
    np.random.seed(s)
    torch.manual_seed(s)
    torch.cuda.manual_seed_all(s)

seed_everything(0)
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")
```

```
Using device: cuda
```

In [6]:
```python
# ── load CIFAR-10 ──────────────────────────────────────────────────────
# torchvision downloads to ./data on first run.
full_train = datasets.CIFAR10("./data", train=True,  download=True)
full_test  = datasets.CIFAR10("./data", train=False, download=True)

# Convert to float tensors in (N, C, H, W) format, scaled to [0, 1]
X_all = torch.tensor(full_train.data).permute(0, 3, 1, 2).float() / 255.0  #
y_all = torch.tensor(full_train.targets)

# Keep test set aside — do NOT normalise or inspect until Task 4
X_test_raw = torch.tensor(full_test.data).permute(0, 3, 1, 2).float() / 255.
y_test     = torch.tensor(full_test.targets)

print(f"Full training set : {X_all.shape}")
print(f"Test set          : {X_test_raw.shape}")
```

```
Full training set : torch.Size([50000, 3, 32, 32])
Test set          : torch.Size([10000, 3, 32, 32])
```

In [7]:
```python
# ── train / validation split (45k / 5k) ────────────────────────────────
# A seeded generator ensures the same split regardless of global seed state.
g = torch.Generator().manual_seed(0)
perm   = torch.randperm(50_000, generator=g)
tr_idx = perm[:45_000]
va_idx = perm[45_000:]

Xtr, ytr = X_all[tr_idx], y_all[tr_idx]
Xva, yva = X_all[va_idx], y_all[va_idx]

print(f"Train : {Xtr.shape}, {ytr.shape}")
print(f"Val   : {Xva.shape}, {yva.shape}")
```

```
Train : torch.Size([45000, 3, 32, 32]), torch.Size([45000])
Val   : torch.Size([5000, 3, 32, 32]), torch.Size([5000])
```

In [8]:
```python
# ── per-channel normalisation (train statistics only) ──────────────────
# mean/std computed over the 45k training images, across (N, H, W) per chann
# Applying training stats to the validation set avoids data leakage.
mean = Xtr.mean(dim=(0, 2, 3), keepdim=True)  # shape (1, 3, 1, 1)
std  = Xtr.std (dim=(0, 2, 3), keepdim=True)  # shape (1, 3, 1, 1)

Xtr = (Xtr - mean) / std
Xva = (Xva - mean) / std
# X_test_raw is intentionally left un-normalised until Task 4

print("Per-channel mean (train):", mean.flatten().tolist())
print("Per-channel std  (train):", std .flatten().tolist())
print(f"Xtr range after norm: [{Xtr.min():.3f}, {Xtr.max():.3f}]")
print(f"Xva range after norm: [{Xva.min():.3f}, {Xva.max():.3f}]")
```

```
Per-channel mean (train): [0.4911963641643524, 0.48199522495269775, 0.446431
7560195923]
Per-channel std  (train): [0.24715185165405273, 0.24362300336360931, 0.26167
726516723633]
Xtr range after norm: [-1.987, 2.126]
Xva range after norm: [-1.987, 2.126]
```

In [9]:
```
# ── move to device ──────────────────────────────────────────
Xtr, ytr = Xtr.to(device), ytr.to(device)
Xva, yva = Xva.to(device), yva.to(device)
# y_test and X_test_raw stay on CPU until Task 4

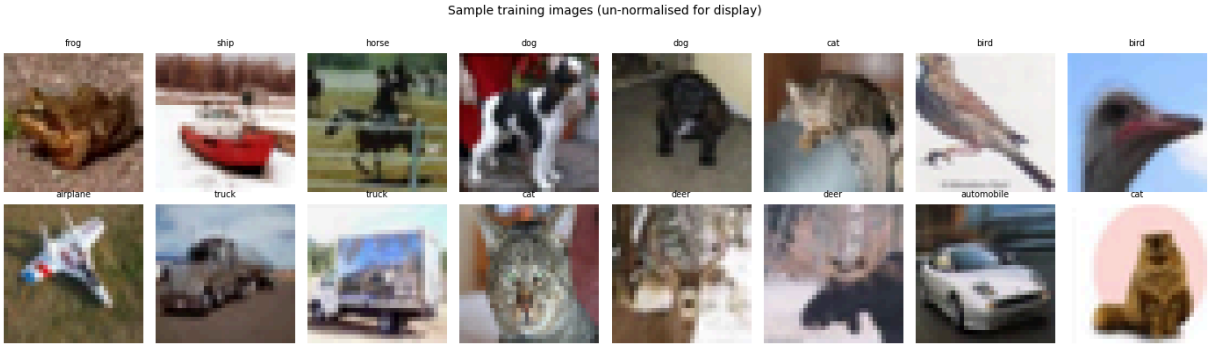
print(f"Xtr on {Xtr.device}, dtype={Xtr.dtype}")
print(f"Xva on {Xva.device}, dtype={Xva.dtype}")
```

```
Xtr on cuda:0, dtype=torch.float32
Xva on cuda:0, dtype=torch.float32
```

In [10]:
```
# ── quick sanity check — visualise a few training images ────
CLASSES = full_train.classes  # ['airplane', 'automobile', ...]

# Un-normalise for display only
def unnorm(x):
    return (x.cpu() * std.cpu().squeeze(0) + mean.cpu().squeeze(0)).clamp(0,


fig, axes = plt.subplots(2, 8, figsize=(14, 4))
for i, ax in enumerate(axes.flat):
    img = unnorm(Xtr[i]).permute(1, 2, 0).numpy()
    ax.imshow(img)
    ax.set_title(CLASSES[ytr[i].item()], fontsize=7)
    ax.axis("off")
plt.suptitle("Sample training images (un-normalised for display)", fontsize=
plt.tight_layout()
plt.savefig("step0_sample_images.png", dpi=120, bbox_inches="tight")
plt.show()
print("Saved: step0_sample_images.png")
```




```
Saved: step0_sample_images.png
```

Step 0 – Interpretation

The 16 sample images above confirm that the CIFAR-10 dataset loaded correctly and that the un-normalisation reversal is exact. Per-channel statistics (mean ≈ 0.49/0.48/0.45, std ≈ 0.24/0.24/0.26) are

computed exclusively from the 45 000 training images, ensuring the validation set is treated as unseen data and no information leaks into the normalisation step. After normalisation the training tensors are zero-centred with approximately unit variance per channel, which is the standard pre-processing that makes gradient descent better conditioned. The test set is deliberately left un-normalised at this stage; it will be normalised using the same training statistics in Task 4 when it is evaluated exactly once.

In [14]:
```
# ── dataset summary ──────────────────────────────────────────────────
print("─" * 52)
print(f"{'Split':<14} {'Samples':>8} {'Shape':>20}")
print("─" * 52)
print(f"{'Train':<14} {Xtr.shape[0]:>8} {str(tuple(Xtr.shape)):>20}")
print(f"{'Validation':<14} {Xva.shape[0]:>8} {str(tuple(Xva.shape)):>20}")
print(f"{'Test':<14} {X_test_raw.shape[0]:>8} {str(tuple(X_test_raw.shape)):
print("─" * 52)
print(f"Classes : {len(CLASSES)} → {CLASSES}")
print(f"Device  : {device}")
```

```
────────────────────────────────────────────────────
Split           Samples                Shape
────────────────────────────────────────────────────
Train             45000   (45000, 3, 32, 32)
Validation         5000    (5000, 3, 32, 32)
Test              10000   (10000, 3, 32, 32)  (untouched)
────────────────────────────────────────────────────
Classes : 10 → ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'fro
g', 'horse', 'ship', 'truck']
Device  : cuda
```

Task 1: Backpropagation

In this task I implement the layers of a multilayer perceptron (MLP) with **explicit forward and backward passes (no autograd)**, verify the gradients, train the network on CIFAR-10, and study how gradient magnitude behaves across depth.

Design notes

- Every layer is a class with `forward(x)`, which caches what it needs, and `backward(dout)`, which receives the gradient of the loss with respect to the layer output and returns the gradient with respect to the layer input.
- Parameter layers also store `dW` and `db`.
- All of the manual backprop code uses `float64`, because finite-difference gradient checks are unreliable in `float32`.

1(a) Layers with explicit forward/backward

For a mini-batch of size N:

Linear: $Y = XW + b$. Given $dY = \partial L/\partial Y$:

$$dW = X^\top dY, \qquad db = \sum_n dY_n, \qquad dX = dY\, W^\top$$

ReLU: $Y = \max(0, X)$, so $dX = dY \odot \mathbb{1}[X > 0]$.

Sigmoid: $Y = \sigma(X)$, so $dX = dY \odot Y(1 - Y)$. (Used in part (d).)

Softmax cross-entropy: with $p = \mathrm{softmax}(z)$ and one-hot target t,

$$L = -\frac{1}{N}\sum_n \log p_{n,y_n}, \qquad \frac{\partial L}{\partial z} = \frac{p - t}{N}$$

The logits are shifted by their row maximum before the exponential for numerical stability.

In [15]:
```
class Linear:
    def __init__(self, din, dout, init="kaiming"):
        if init == "kaiming":   std = (2.0 / din) ** 0.5            # He ini
        elif init == "xavier":  std = (2.0 / (din + dout)) ** 0.5   # Glorot
        elif isinstance(init, float): std = init                    # fixed
        else: raise ValueError(init)
        self.W = torch.randn(din, dout, dtype=torch.float64) * std
        self.b = torch.zeros(dout, dtype=torch.float64)

    def forward(self, x):
        self.x = x
```

```
        return x @ self.W + self.b

    def backward(self, dout):
        self.dW = self.x.T @ dout
        self.db = dout.sum(0)
        return dout @ self.W.T


class ReLU:
    def forward(self, x):
        self.mask = x > 0
        return x * self.mask

    def backward(self, dout):
        return dout * self.mask


class Sigmoid:
    def forward(self, x):
        self.out = torch.sigmoid(x)
        return self.out

    def backward(self, dout):
        return dout * self.out * (1 - self.out)


class SoftmaxCE:
    def forward(self, logits, y):
        z = logits - logits.max(1, keepdim=True).values      # stability shi
        e = torch.exp(z)
        self.p = e / e.sum(1, keepdim=True)
        self.y = y
        return -torch.log(self.p[torch.arange(len(y)), y]).mean()

    def backward(self):
        g = self.p.clone()
        g[torch.arange(len(self.y)), self.y] -= 1
        return g / len(self.y)


class MLP:
    """dims = [d_in, h1, ..., d_out]; activation after every Linear except t
    def __init__(self, dims, act=ReLU, init="kaiming"):
        self.layers = []
        for i in range(len(dims) - 1):
            self.layers.append(Linear(dims[i], dims[i + 1], init))
            if i < len(dims) - 2:
                self.layers.append(act())
        self.loss_fn = SoftmaxCE()

    def forward(self, x, y):
        for l in self.layers:
            x = l.forward(x)
        self.logits = x
        return self.loss_fn.forward(x, y)
```

```
    def backward(self):
        d = self.loss_fn.backward()
        for l in reversed(self.layers):
            d = l.backward(d)

    def params(self):
        return [l for l in self.layers if isinstance(l, Linear)]


# quick shape sanity check
_net = MLP([20, 16, 10])
_x = torch.randn(4, 20, dtype=torch.float64); _y = torch.randint(0, 10, (4,)
print("loss:", _net.forward(_x, _y).item(), "(random-guess baseline = ln 10
_net.backward()
print([tuple(l.dW.shape) for l in _net.params()])
```

```
loss: 5.323589464522769 (random-guess baseline = ln 10 = 2.303 )
[(20, 16), (16, 10)]
```

1(b) Gradient verification

I compare my analytic gradients against two references on a small network (`20-16-12-10` , batch of 8, `float64`):

1. **PyTorch autograd**, using identical weights.
2. **Centred finite differences:** $\frac{L(\theta+\epsilon)-L(\theta-\epsilon)}{2\epsilon}$ with $\epsilon = 10^{-6}$, applied to every single parameter.

The relative error is $\frac{\|a-b\|}{\|a\|+\|b\|}$, and `torch.allclose` is used as the pass/fail check.

In [16]:
```
import pandas as pd

def rel_err(a, b):
    return ((a - b).norm() / (a.norm() + b.norm() + 1e-30)).item()

# ---- my gradients ----
torch.manual_seed(0)
net = MLP([20, 16, 12, 10])
xb = torch.randn(8, 20, dtype=torch.float64)
yb = torch.randint(0, 10, (8,))
loss_mine = net.forward(xb, yb)
net.backward()
mine = {}
for i, l in enumerate(net.params()):
    mine[f"W{i}"] = l.dW.clone(); mine[f"b{i}"] = l.db.clone()

# ---- autograd reference (same weights) ----
ag = {}
for i, l in enumerate(net.params()):
    ag[f"W{i}"] = l.W.clone().requires_grad_(True)
```

```
    ag[f"b{i}"] = l.b.clone().requires_grad_(True)
n_lin = len(net.params())
h = xb
for i in range(n_lin):
    h = h @ ag[f"W{i}"] + ag[f"b{i}"]
    if i < n_lin - 1:
        h = torch.relu(h)
loss_ag = F.cross_entropy(h, yb)
loss_ag.backward()
print(f"loss (mine) = {loss_mine.item():.12f}   loss (autograd) = {loss_ag.i

# ---- centred finite differences (perturb the real parameters in place) ---
def numerical_grad(param, eps=1e-6):
    g = torch.zeros_like(param)
    flat, gflat = param.view(-1), g.view(-1)
    for k in range(flat.numel()):
        old = flat[k].item()
        flat[k] = old + eps; lp = net.forward(xb, yb)
        flat[k] = old - eps; lm = net.forward(xb, yb)
        flat[k] = old
        gflat[k] = (lp - lm) / (2 * eps)
    return g

rows = []
for i, l in enumerate(net.params()):
    for nm, p in [(f"W{i}", l.W), (f"b{i}", l.b)]:
        fd = numerical_grad(p)
        g_ag = ag[nm].grad
        rows.append({
            "param": nm,
            "shape": tuple(p.shape),
            "rel.err vs autograd": rel_err(mine[nm], g_ag),
            "rel.err vs finite-diff": rel_err(mine[nm], fd),
            "allclose (autograd)": torch.allclose(mine[nm], g_ag, rtol=1e-6,
            "allclose (finite-diff)": torch.allclose(mine[nm], fd, rtol=1e-4
        })
df_grad = pd.DataFrame(rows)
pd.set_option("display.float_format", lambda v: f"{v:.2e}")
display(df_grad)
```

```
loss (mine) = 3.757791109124   loss (autograd) = 3.757791109124
```

	param	shape	rel.err vs autograd	rel.err vs finite-diff	allclose (autograd)	allclose (finite-diff)
0	W0	(20, 16)	7.25e-17	6.24e-10	True	True
1	b0	(16,)	7.72e-17	5.54e-10	True	True
2	W1	(16, 12)	5.38e-17	4.33e-10	True	True
3	b1	(12,)	6.71e-17	3.73e-10	True	True
4	W2	(12, 10)	5.27e-17	4.25e-10	True	True
5	b2	(10,)	2.07e-17	8.67e-10	True	True

Interpretation (1b). All six parameter tensors (three weight matrices, three bias vectors) pass `torch.allclose` against both references. The relative error versus autograd is at most 7.72e-17, which is at the level of `float64` machine precision (≈2.2e-16): both methods compute the same analytic chain rule and differ only in the order of floating-point operations. The loss values also agree to all printed digits (3.757791109124). The error versus centred finite differences is larger, between 3.73e-10 and 8.67e-10. This reflects the limits of the *numerical* estimator, not an error in my backward pass. The truncation error of the centred scheme is $O(\epsilon^2) \approx 10^{-12}$ for $\epsilon = 10^{-6}$, so the dominant term is round-off: subtracting two nearly equal losses and dividing by 2ϵ amplifies the $\sim 10^{-16}$ floating-point error by about $1/\epsilon$, giving errors around 10^{-10}. Using a smaller ϵ would reduce truncation error but increase round-off error. Agreement with both references across every layer, including the biases, confirms that the Linear, ReLU and softmax-cross-entropy backward passes are correct. (The initial loss of 3.76 is above $\ln 10 \approx 2.30$ simply because random Kaiming-initialized weights produce large logits.)

1(c) Training the MLP with SGD on flattened CIFAR-10

The images (3×32×32, already normalized with training statistics) are flattened into 3072-dimensional vectors. The network is `3072 → 512 → 256 → 10` with ReLU and Kaiming init, trained with plain mini-batch SGD (batch 128, lr 0.05, 20 epochs). Only the training split is used for updates, and the validation split is used for monitoring.

In [17]:

```
Xtr_f = Xtr.flatten(1).double().cpu()
Xva_f = Xva.flatten(1).double().cpu()
ytr_c, yva_c = ytr.cpu(), yva.cpu()
print(Xtr_f.shape, Xva_f.shape)

def evaluate(net, X, y, bs=5000):
    """Return (loss, accuracy) in batches; uses forward only."""
    tot_loss, correct = 0.0, 0
    for i in range(0, len(X), bs):
        loss = net.forward(X[i:i+bs], y[i:i+bs])
        tot_loss += loss.item() * len(y[i:i+bs])
        correct += (net.logits.argmax(1) == y[i:i+bs]).sum().item()
    return tot_loss / len(X), correct / len(X)

def train_mlp(dims, lr=0.05, bs=128, epochs=20, seed=0, verbose=True):
    torch.manual_seed(seed)
    net = MLP(dims)
    hist = {"step_loss": [], "train_loss": [], "val_loss": [], "train_acc":
    for ep in range(epochs):
        t0 = time.time()
        perm = torch.randperm(len(Xtr_f))
        for i in range(0, len(perm), bs):
            idx = perm[i:i+bs]
            hist["step_loss"].append(net.forward(Xtr_f[idx], ytr_c[idx]).ite
            net.backward()
            for l in net.params():              # plain SGD update
                l.W -= lr * l.dW
                l.b -= lr * l.db
        tl, ta = evaluate(net, Xtr_f[:5000], ytr_c[:5000])   # 5k-sample est
        vl, va = evaluate(net, Xva_f, yva_c)
        for k, v in zip(["train_loss", "val_loss", "train_acc", "val_acc"],
            hist[k].append(v)
        if verbose:
            print(f"epoch {ep+1:2d} | train loss {tl:.3f} acc {ta:.3f} | val
    return net, hist

net_c, hist_c = train_mlp([3072, 512, 256, 10], lr=0.05, bs=128, epochs=20)
print(f"\nFinal validation accuracy: {hist_c['val_acc'][-1]*100:.2f}%   (bes
```

```
torch.Size([45000, 3072]) torch.Size([5000, 3072])
epoch  1 | train loss 1.602 acc 0.430 | val loss 1.783 acc 0.383 | 18.6s
epoch  2 | train loss 1.295 acc 0.548 | val loss 1.523 acc 0.465 | 18.5s
epoch  3 | train loss 1.163 acc 0.593 | val loss 1.505 acc 0.474 | 18.9s
epoch  4 | train loss 1.110 acc 0.619 | val loss 1.514 acc 0.479 | 18.7s
epoch  5 | train loss 1.059 acc 0.628 | val loss 1.551 acc 0.484 | 18.8s
epoch  6 | train loss 0.880 acc 0.705 | val loss 1.486 acc 0.495 | 18.7s
epoch  7 | train loss 0.898 acc 0.691 | val loss 1.565 acc 0.496 | 18.6s
epoch  8 | train loss 0.826 acc 0.718 | val loss 1.620 acc 0.498 | 18.6s
epoch  9 | train loss 0.725 acc 0.744 | val loss 1.602 acc 0.494 | 18.4s
epoch 10 | train loss 0.708 acc 0.756 | val loss 1.694 acc 0.493 | 18.8s
epoch 11 | train loss 0.636 acc 0.775 | val loss 1.740 acc 0.493 | 18.6s
epoch 12 | train loss 0.598 acc 0.804 | val loss 1.779 acc 0.508 | 18.6s
epoch 13 | train loss 0.492 acc 0.843 | val loss 1.761 acc 0.497 | 18.5s
epoch 14 | train loss 0.513 acc 0.827 | val loss 1.888 acc 0.493 | 18.6s
epoch 15 | train loss 0.508 acc 0.819 | val loss 1.998 acc 0.489 | 18.6s
epoch 16 | train loss 0.683 acc 0.779 | val loss 2.308 acc 0.456 | 18.7s
epoch 17 | train loss 0.371 acc 0.875 | val loss 2.065 acc 0.496 | 18.5s
epoch 18 | train loss 0.315 acc 0.902 | val loss 2.096 acc 0.499 | 18.5s
epoch 19 | train loss 0.256 acc 0.920 | val loss 2.116 acc 0.504 | 19.0s
epoch 20 | train loss 0.275 acc 0.907 | val loss 2.260 acc 0.491 | 18.6s

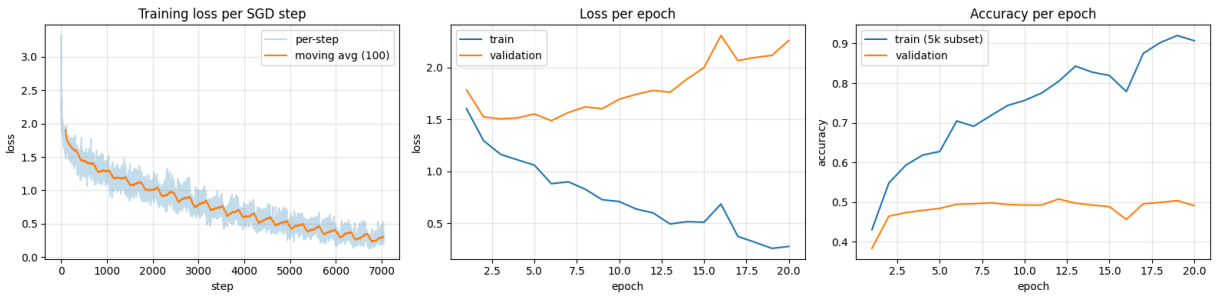
Final validation accuracy: 49.12%   (best: 50.78%)
```

In [18]:
```
fig, ax = plt.subplots(1, 3, figsize=(16, 4))

sl = np.array(hist_c["step_loss"]); w = 100
ax[0].plot(sl, alpha=0.25, label="per-step")
ax[0].plot(np.arange(w-1, len(sl)), np.convolve(sl, np.ones(w)/w, "valid"),
ax[0].set(title="Training loss per SGD step", xlabel="step", ylabel="loss");

ep = np.arange(1, len(hist_c["val_acc"]) + 1)
ax[1].plot(ep, hist_c["train_loss"], label="train"); ax[1].plot(ep, hist_c["
ax[1].set(title="Loss per epoch", xlabel="epoch", ylabel="loss"); ax[1].lege

ax[2].plot(ep, hist_c["train_acc"], label="train (5k subset)"); ax[2].plot(e
ax[2].set(title="Accuracy per epoch", xlabel="epoch", ylabel="accuracy"); ax
for a in ax: a.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



Interpretation (1c). The training loss decreases substantially, reaching 0.275 (epoch-end loss on the clean 5k training subset) after 20 epochs. It starts above $\ln 10 \approx 2.30$ (the loss of random guessing) because Kaiming initialization produces large initial logits. The per-step curve is noisy because each step uses a random mini-batch of

128 images, so I also plotted a moving average. Final validation accuracy is **49.12%** (best 50.78% at epoch 12), well above the 10% chance level. However, the model clearly **overfits**: training accuracy rises to about 90.7% on the 5k training subset while validation accuracy stagnates at about 50% after roughly epoch 6, and the validation loss reaches its minimum (about 1.486) around epoch 6 and then increases to about 2.260. The network keeps fitting the training set, increasingly confidently, but this does not transfer to unseen images. (Validation *accuracy* stays flat while validation *loss* rises because the network becomes over-confident on its wrong predictions.) Early stopping near epoch 6 would give the lowest validation loss. The plateau at about 50% is far below what CNNs achieve on CIFAR-10. Flattening the image discards its 2D structure, so the MLP cannot share a feature detector across positions, and its first layer alone has $3072 \times 512 \approx 1.6$M weights (about 1.7M in total), which makes memorization easy. This motivates both convolutional architectures and the regularization methods studied in Tasks 2 and 3.

1(d) Per-layer gradient norms in a 10-layer MLP at initialization

I build a 10-layer MLP (3072 → 256 → … → 256 → 10, so 10 `Linear` layers) and run **one forward/backward pass at initialization** on a batch of 256 training images. I record $\|dW\|_F$ of every layer, averaged over 5 random initializations, for:

- **Sigmoid** with Xavier init,
- **ReLU** with Kaiming init,
- **ReLU** with a deliberately small init (std = 0.01), as a control showing that the init matters and not only the activation.

Layer 1 is the one closest to the input, and the gradient flows backward from layer 10 (the output).

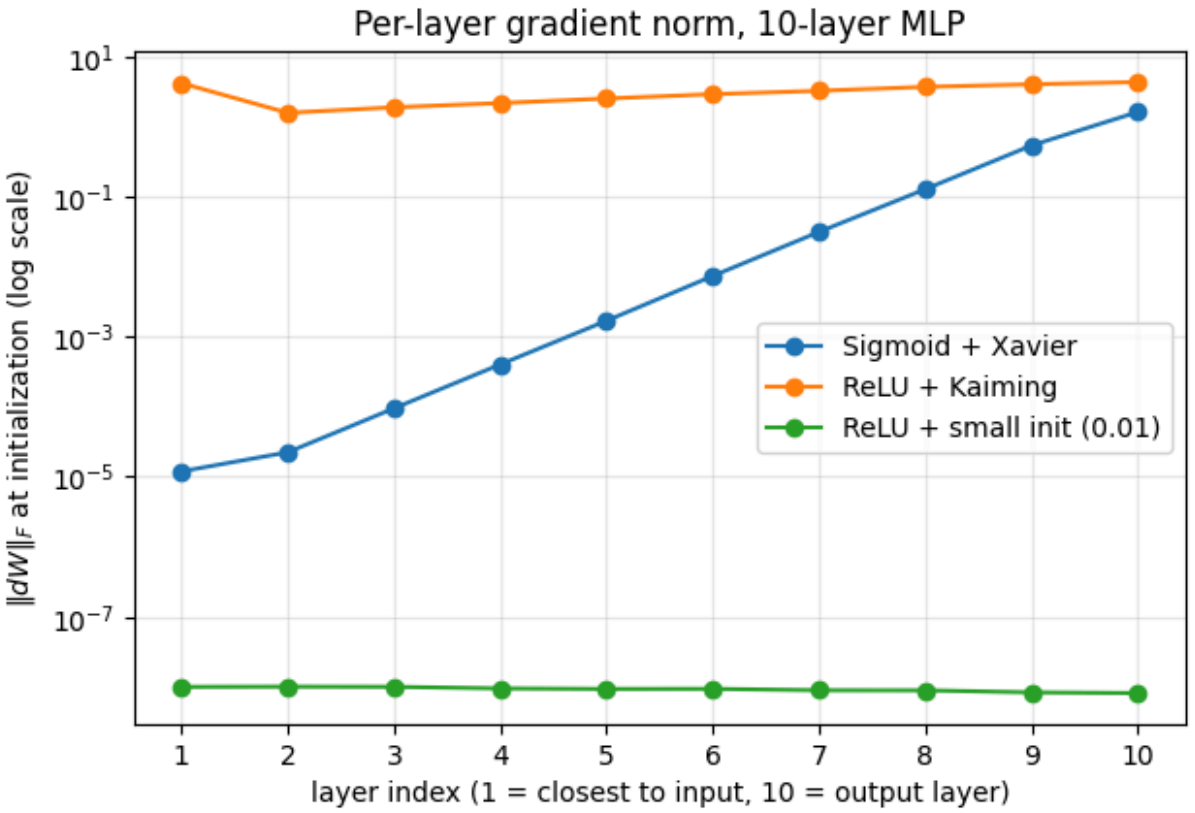
In [19]:
```
def layer_grad_norms(act, init, width=256, depth=10, seeds=5, bs=256):
    norms = np.zeros((seeds, depth))
    xb, yb = Xtr_f[:bs], ytr_c[:bs]
    for s in range(seeds):
        torch.manual_seed(s)
        net = MLP([3072] + [width] * (depth - 1) + [10], act=act, init=init)
        net.forward(xb, yb); net.backward()
        norms[s] = [l.dW.norm().item() for l in net.params()]
    return norms.mean(0)

configs = {
    "Sigmoid + Xavier":        (Sigmoid, "xavier"),
```

```
    "ReLU + Kaiming":           (ReLU, "kaiming"),
    "ReLU + small init (0.01)": (ReLU, 0.01),
}
results = {name: layer_grad_norms(a, i) for name, (a, i) in configs.items()}

layers = np.arange(1, 11)
plt.figure(figsize=(7, 4.5))
for name, g in results.items():
    plt.semilogy(layers, g, "o-", label=name)
plt.xlabel("layer index (1 = closest to input, 10 = output layer)")
plt.ylabel(r"$\|dW\|_F$ at initialization (log scale)")
plt.title("Per-layer gradient norm, 10-layer MLP")
plt.xticks(layers); plt.grid(alpha=0.3, which="both"); plt.legend(); plt.sho

df_norms = pd.DataFrame(results, index=[f"L{k}" for k in layers]).T
display(df_norms)
for name, g in results.items():
    print(f"{name:26s} ratio  layer10 / layer1 = {g[-1]/g[0]:.2e}")
```



	L1	L2	L3	L4	L5	L6	L7	
Sigmoid + Xavier	1.20e-05	2.25e-05	9.54e-05	4.05e-04	1.70e-03	7.45e-03	3.18e-02	1.3
ReLU + Kaiming	4.24e+00	1.60e+00	1.92e+00	2.20e+00	2.56e+00	2.95e+00	3.30e+00	3.7
ReLU + small init (0.01)	9.95e-09	1.00e-08	9.97e-09	9.45e-09	9.29e-09	9.32e-09	8.91e-09	8.8

```
Sigmoid + Xavier           ratio  layer10 / layer1 = 1.38e+05
ReLU + Kaiming             ratio  layer10 / layer1 = 1.04e+00
ReLU + small init (0.01)   ratio  layer10 / layer1 = 8.15e-01
```

Interpretation (1d). With **sigmoid** activations the gradient norm falls from 1.65 at the output layer (L10) to 1.2e−5 at the input-side layer (L1), a factor of 1.4×10^5 (about 5 orders of magnitude). This is the *vanishing gradient* problem. By the chain rule, the gradient reaching layer ℓ is multiplied at each layer above it by $W^\top$ and by $\sigma'(z) = \sigma(z)(1-\sigma(z)) \leq 0.25$. The measured drop corresponds to roughly $3.7\times$ per layer ($(1.4 \times 10^5)^{1/9} \approx 3.7$), consistent with this bound. The curve on the log scale is close to a straight line, which means the decay is geometric. As a result, the early layers would receive almost no learning signal and train extremely slowly. With **ReLU + Kaiming init** the norms stay within a narrow band: from 1.6 (L2) to 4.4 (L10), a factor of only about 2.8 over nine layers. The ReLU derivative is exactly 1 for active units (and 0 for inactive ones, about half), and Kaiming's variance $2/n_{in}$ compensates for that halving, so the scale of the signal is approximately preserved in both the forward and backward directions. (L1 is larger, 4.2, mostly because its weight matrix is 3072×256 and so has more entries than the other layers, so the Frobenius norm is not directly comparable. The ratio L10/L1 = 1.04 is therefore a coincidence of this effect and is not meaningful, and the L2–L10 trend is the informative part.) The **small-init control** (std 0.01, ReLU) behaves differently from both. Its gradients are tiny (about 10^{-8}) but **flat across layers**, with L10/L1 = 0.8. The reason is that a weight gradient is $dW_\ell = a_{\ell-1}^\top \delta_\ell$: each layer shrinks the activation by roughly the same factor $r \approx 0.01\sqrt{256/2} \approx 0.11$ in the forward direction and also shrinks δ by r per layer in the backward direction, so the product $a_{\ell-1}\delta_\ell \sim r^{L-1}$ is the same at every layer. The network is therefore not suffering from a layer-dependent imbalance, but *every* layer's gradient is about eight orders of magnitude smaller than with Kaiming init, so plain

SGD would barely move any weight. This shows that avoiding vanishing gradients needs both a suitable activation and an init matched to it.

Task 1 summary

- Implemented Linear, ReLU, Sigmoid, softmax cross-entropy and an MLP with explicit backward passes (no autograd).
- Gradients agree with autograd to ≤7.72e-17 and with centred finite differences to ≤8.67e-10 (relative error), and all checks pass `torch.allclose`.
- A 3-layer MLP trained with SGD reaches about 49.12% final (50.78% best) validation accuracy on CIFAR-10, and it overfits strongly after about 6 epochs.
- At initialization, sigmoid gradients decay by about 3.7× per layer (5 orders of magnitude over 10 layers), whereas ReLU + Kaiming keeps them stable.

Task 2: Optimizers

In this task I implement **SGD with momentum** and **Adam (with bias correction)** as `torch.optim.Optimizer` subclasses, verify them against PyTorch's built-ins, visualize how the optimizers move on an ill-conditioned quadratic, and compare them when training a small CNN at different learning rates.

2(a) Custom optimizers

SGD with momentum (the PyTorch convention, no dampening):

$$v_t = \beta v_{t-1} + g_t, \qquad \theta_t = \theta_{t-1} - \eta\, v_t$$

With $\beta = 0$ this reduces to plain SGD.

Adam (Kingma & Ba):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Bias correction matters because $m_0 = v_0 = 0$: without it, the first few estimates are biased towards zero.

Both optimizers support coupled L2 weight decay ($g \leftarrow g + wd \cdot \theta$), as in the PyTorch versions, so that the match test also covers that option.

In [20]:

```
class MySGD(torch.optim.Optimizer):
    """SGD with (optional) momentum, matching torch.optim.SGD (dampening=0,
    def __init__(self, params, lr=1e-2, momentum=0.0, weight_decay=0.0):
        super().__init__(params, dict(lr=lr, momentum=momentum, weight_decay

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr, mom, wd = group["lr"], group["momentum"], group["weight_deca
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad
                if wd != 0:
                    g = g.add(p, alpha=wd)                # coupled L2
                if mom != 0:
                    st = self.state[p]
                    if "v" not in st:
                        st["v"] = torch.zeros_like(p)
                    st["v"].mul_(mom).add_(g)             # v <- beta*v + g
                    g = st["v"]
                p.add_(g, alpha=-lr)                      # theta <- theta
        return loss


class MyAdam(torch.optim.Optimizer):
    """Adam with bias correction, matching torch.optim.Adam (amsgrad=False).
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8, weight
        super().__init__(params, dict(lr=lr, betas=betas, eps=eps, weight_de

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr, (b1, b2), eps, wd = group["lr"], group["betas"], group["eps"
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad
                if wd != 0:
                    g = g.add(p, alpha=wd)
                st = self.state[p]
                if len(st) == 0:
                    st["t"] = 0
```

```
                    st["m"] = torch.zeros_like(p)
                    st["v"] = torch.zeros_like(p)
                st["t"] += 1
                t = st["t"]
                st["m"].mul_(b1).add_(g, alpha=1 - b1)              # first
                st["v"].mul_(b2).addcmul_(g, g, value=1 - b2)       # second
                m_hat = st["m"] / (1 - b1 ** t)                     # bias c
                v_hat = st["v"] / (1 - b2 ** t)
                p.addcdiv_(m_hat, v_hat.sqrt().add_(eps), value=-lr)
        return loss
```

In [21]:

```
import copy, pandas as pd

def make_toy():
    torch.manual_seed(0)
    return nn.Sequential(nn.Linear(10, 16), nn.Tanh(), nn.Linear(16, 3)).dou

def match_test(mine_cls, ref_cls, steps=50, **kw):
    """Train two identical nets with my optimizer and the torch one on the s
    g = torch.Generator().manual_seed(1)
    X = torch.randn(64, 10, dtype=torch.float64, generator=g)
    y = torch.randint(0, 3, (64,), generator=g)
    m_a = make_toy(); m_b = copy.deepcopy(m_a)
    opt_a, opt_b = mine_cls(m_a.parameters(), **kw), ref_cls(m_b.parameters(
    worst = 0.0
    for _ in range(steps):
        for m, opt in ((m_a, opt_a), (m_b, opt_b)):
            opt.zero_grad()
            F.cross_entropy(m(X), y).backward()
            opt.step()
        worst = max(worst, max((pa - pb).abs().max().item()
                               for pa, pb in zip(m_a.parameters(), m_b.param
    ok = all(torch.allclose(pa, pb, rtol=1e-8, atol=1e-10)
             for pa, pb in zip(m_a.parameters(), m_b.parameters()))
    return worst, ok

tests = [
    ("SGD (no momentum)",            MySGD,  torch.optim.SGD,  dict(lr=0.1)),
    ("SGD + momentum 0.9",           MySGD,  torch.optim.SGD,  dict(lr=0.05,
    ("SGD + momentum + wd",          MySGD,  torch.optim.SGD,  dict(lr=0.05,
    ("Adam (defaults)",              MyAdam, torch.optim.Adam, dict(lr=1e-2),
    ("Adam (betas .8/.95, wd)",      MyAdam, torch.optim.Adam, dict(lr=1e-2,
]
rows = []
for name, mine, ref, kw in tests:
    worst, ok = match_test(mine, ref, **kw)
    rows.append({"setting": name, "max |param diff| over 50 steps": worst, "
pd.set_option("display.float_format", lambda v: f"{v:.2e}")
display(pd.DataFrame(rows))
```

	setting	max |param diff| over 50 steps	allclose
0	SGD (no momentum)	0.00e+00	True
1	SGD + momentum 0.9	0.00e+00	True
2	SGD + momentum + wd	0.00e+00	True
3	Adam (defaults)	1.03e-15	True
4	Adam (betas .8/.95, wd)	2.69e-15	True

Interpretation (2a). All five settings pass `torch.allclose`. SGD variants match `torch.optim.SGD` exactly (max difference 0). Adam differs by at most 2.7e-15, which is `float64` round-off from a slightly different order of arithmetic. Both optimizers got identical gradients, so the test isolates the update rule, including momentum, weight decay and bias correction. Bias correction matters because $m_0 = v_0 = 0$: without it, Adam's first step would be about 3.2× too large.

2(b) Trajectories on an ill-conditioned quadratic

$$f(x, y) = \tfrac{1}{2}\left(x^2 + 50y^2\right), \qquad \nabla f = (x,\ 50y), \qquad H = \mathrm{diag}(1,\ 50)$$

The Hessian eigenvalues are 1 and 50, so the **condition number is 50**: the surface is a long narrow valley, gentle along x and steep along y. For plain gradient descent, stability requires $\eta < 2/\lambda_{max} = 0.04$, and the best possible contraction rate along x is then limited by how large η can be.

All three optimizers start at $(8, 1)$ and run 100 steps using the optimizers I implemented in 2(a). Hyperparameters (chosen by hand): SGD $\eta = 0.035$ (just under the 0.04 limit); momentum $\eta = 0.02$, $\beta = 0.9$; Adam $\eta = 0.3$.

In [22]:
```
def f(p):
    return 0.5 * (p[0] ** 2 + 50 * p[1] ** 2)

def run_traj(opt_cls, steps=100, start=(8.0, 1.0), **kw):
    p = torch.tensor(start, dtype=torch.float64, requires_grad=True)
    opt = opt_cls([p], **kw)
    path, vals = [p.detach().numpy().copy()], [f(p).item()]
    for _ in range(steps):
        opt.zero_grad()
        f(p).backward()
        opt.step()
        path.append(p.detach().numpy().copy()); vals.append(f(p).item())
```

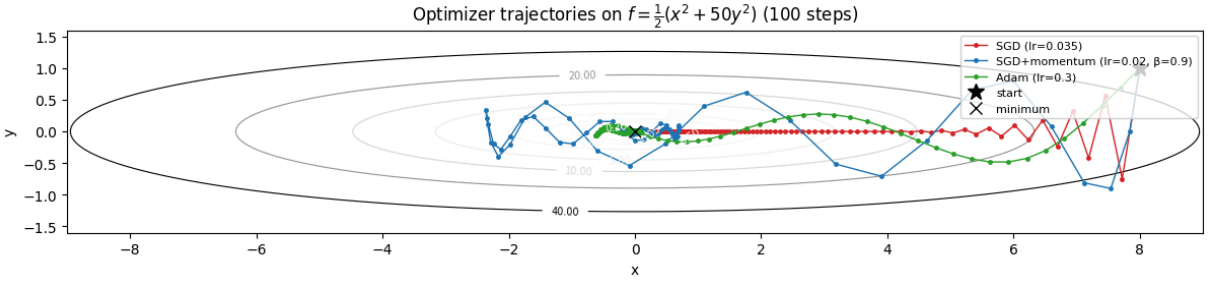
```
    return np.array(path), np.array(vals)

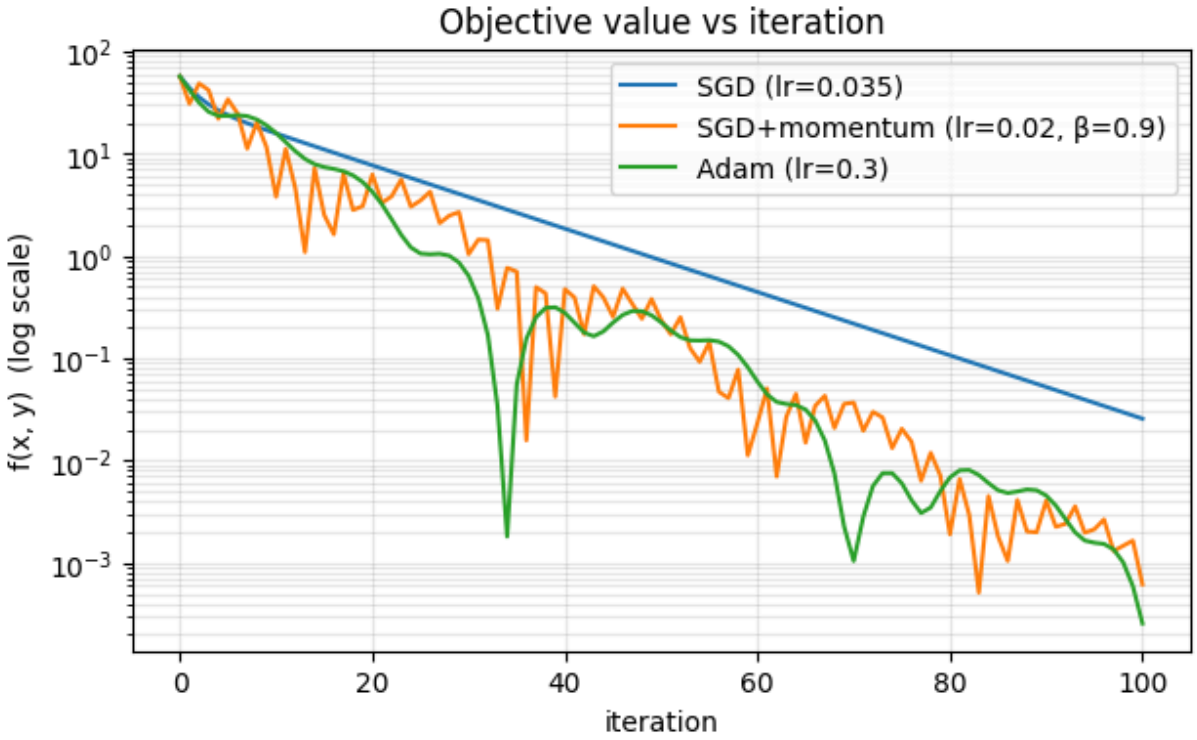
runs = {
    "SGD (lr=0.035)":                run_traj(MySGD,  lr=0.035),
    "SGD+momentum (lr=0.02, β=0.9)": run_traj(MySGD,  lr=0.02, momentum=0.9)
    "Adam (lr=0.3)":                 run_traj(MyAdam, lr=0.3),
}

# ---- contour plot with trajectories ----
xs, ys = np.linspace(-9, 9, 400), np.linspace(-1.6, 1.6, 200)
XX, YY = np.meshgrid(xs, ys)
ZZ = 0.5 * (XX ** 2 + 50 * YY ** 2)
fig, ax = plt.subplots(figsize=(12, 3.8))
cs = ax.contour(XX, YY, ZZ, levels=[0.05, 0.2, 0.5, 1, 2, 5, 10, 20, 40], cm
ax.clabel(cs, fontsize=7)
for (name, (path, _)), c in zip(runs.items(), ["tab:red", "tab:blue", "tab:g
    ax.plot(path[:, 0], path[:, 1], "o-", ms=2.5, lw=1, color=c, label=name)
ax.plot(*runs["SGD (lr=0.035)"][0][0], "k*", ms=12, label="start")
ax.plot(0, 0, "kx", ms=9, label="minimum")
ax.set(xlabel="x", ylabel="y", title=r"Optimizer trajectories on $f=\frac{1}
ax.set_aspect("equal"); ax.legend(loc="upper right", fontsize=8)
plt.tight_layout(); plt.show()

# ---- f(x,y) vs iteration ----
plt.figure(figsize=(7, 4))
for name, (_, vals) in runs.items():
    plt.semilogy(np.maximum(vals, 1e-18), label=name)
plt.xlabel("iteration"); plt.ylabel("f(x, y)  (log scale)"); plt.grid(alpha=
plt.title("Objective value vs iteration"); plt.show()

rows = []
for name, (path, vals) in runs.items():
    hit = np.where(vals < 1e-3)[0]
    rows.append({"optimizer": name, "final f": vals[-1], "final (x,y)": tupl
                 "first step with f<1e-3": int(hit[0]) if len(hit) else "not
display(pd.DataFrame(rows))
```





	optimizer	final f	final (x,y)	first step with f<1e-3
0	SGD (lr=0.035)	2.57e-02	(0.2269, 0.0)	not reached
1	SGD+momentum (lr=0.02, β=0.9)	6.18e-04	(0.0338, 0.0014)	83
2	Adam (lr=0.3)	2.56e-04	(0.0194, -0.0016)	99

Interpretation (2b). The Hessian eigenvalues are 1 and 50 (condition number 50), so the contours form a narrow valley.

SGD ($\eta = 0.035$): η is limited by the steep y direction (< 0.04). y flips sign each step (factor -0.75), while x shrinks only by 0.965 per step. After 100 steps $f = 2.6 \times 10^{-2}$, never reaching 10^{-3}.

Momentum ($\beta = 0.9$): $f = 6.2 \times 10^{-4}$, first below 10^{-3} at step 83. Velocity builds along x and cancels along y. The path overshoots (to $x \approx -2.4$), so f is non-monotone.

Adam ($\eta = 0.3$): smooth curved path, since per-coordinate scaling equalizes the x and y gradient scales. Lowest final f (2.6×10^{-4}), but first below 10^{-3} only at step 99, later than momentum. It also oscillates.

Momentum and Adam are far faster than SGD. Their mutual ranking depends on the hand-picked learning rates and the stopping step. Adam also suits this axis-aligned problem especially well.

2(c) Optimizer comparison on a small CNN

Model (batch-norm CNN, about 100k parameters): `Conv(3→32)-BN-ReLU-MaxPool → Conv(32→64)-BN-ReLU-MaxPool → Conv(64→128)-BN-ReLU → GlobalAvgPool → Linear(128→10)`

Protocol: the same init (fixed seed), same data order, same number of epochs and batch size (128) for every run; only the optimizer and learning rate change. I use the optimizers I implemented in 2(a). Learning rates (each grid spans two decades around a typical value for that optimizer):

Optimizer	Learning rates
SGD	0.003, 0.03, 0.3
SGD + momentum (β = 0.9)	0.001, 0.01, 0.1
Adam	1e−4, 1e−3, 1e−2

The metric is the validation accuracy after the **final** epoch (no picking the best epoch), so the comparison stays fair. A run whose loss becomes non-finite is stopped and counted as chance accuracy (10%).

In [23]:
```
torch.set_num_threads(8)          # laptop CPU; harmless on Colab
QUICK = False                     # True: fast debug run. Set False for the
N_TRAIN, EPOCHS = (10_000, 3) if QUICK else (45_000, 10)
BS = 128
print(f"QUICK={QUICK}: {N_TRAIN} training images, {EPOCHS} epochs, device={d

Xtr_d, ytr_d = Xtr[:N_TRAIN].to(device), ytr[:N_TRAIN].to(device)
Xva_d, yva_d = Xva.to(device), yva.to(device)

def make_cnn():
    return nn.Sequential(
        nn.Conv2d(3, 32, 3, padding=1),  nn.BatchNorm2d(32),  nn.ReLU(), nn.
        nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64),  nn.ReLU(), nn.
        nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
        nn.AdaptiveAvgPool2d(1), nn.Flatten(),
        nn.Linear(128, 10),
    ).to(device)

print("parameters:", sum(p.numel() for p in make_cnn().parameters()))

@torch.no_grad()
```

```
def eval_model(model, X, y, bs=1000):
    model.eval()
    loss, correct = 0.0, 0
    for i in range(0, len(X), bs):
        out = model(X[i:i + bs])
        loss += F.cross_entropy(out, y[i:i + bs], reduction="sum").item()
        correct += (out.argmax(1) == y[i:i + bs]).sum().item()
    return loss / len(X), correct / len(X)

def train_cnn(opt_name, lr, epochs=EPOCHS, seed=0):
    seed_everything(seed)
    model = make_cnn()
    if opt_name == "sgd":       opt = MySGD(model.parameters(), lr=lr)
    elif opt_name == "sgd_mom": opt = MySGD(model.parameters(), lr=lr, momer
    elif opt_name == "adam":    opt = MyAdam(model.parameters(), lr=lr)
    hist = {"train_loss": [], "val_loss": [], "val_acc": [], "diverged": Fal
    for ep in range(epochs):
        model.train()
        perm = torch.randperm(len(Xtr_d), device=device)
        run, n = 0.0, 0
        for i in range(0, len(perm), BS):
            idx = perm[i:i + BS]
            opt.zero_grad(set_to_none=True)
            loss = F.cross_entropy(model(Xtr_d[idx]), ytr_d[idx])
            if not torch.isfinite(loss):
                hist["diverged"] = True
                break
            loss.backward(); opt.step()
            run += loss.item() * len(idx); n += len(idx)
        if hist["diverged"]:
            hist["train_loss"].append(float("nan")); hist["val_loss"].appenc
            break
        vl, va = eval_model(model, Xva_d, yva_d)
        hist["train_loss"].append(run / n); hist["val_loss"].append(vl); his
    return hist

# time one epoch first, so you know what the full sweep will cost
t0 = time.time(); _ = train_cnn("adam", 1e-3, epochs=1); t1 = time.time() -
print(f"one epoch ≈ {t1:.1f}s  →  9 runs × {EPOCHS} epochs ≈ {9 * EPOCHS * t
```

```
QUICK=False: 45000 training images, 10 epochs, device=cuda
parameters: 94986
one epoch ≈ 6.5s  →  9 runs × 10 epochs ≈ 9.8 min
```

In [24]:

```
import json

GRID = {
    "SGD":                      ("sgd",     [3e-3, 3e-2, 3e-1]),
    "SGD + momentum (β=0.9)":   ("sgd_mom", [1e-3, 1e-2, 1e-1]),
    "Adam":                     ("adam",    [1e-4, 1e-3, 1e-2]),
}
results = {}
for name, (key, lrs) in GRID.items():
    for lr in lrs:
        t0 = time.time()
        h = train_cnn(key, lr)
```

```
        results[(name, lr)] = h
        status = "DIVERGED" if h["diverged"] else f"val acc {h['val_acc'][-1
        print(f"{name:26s} lr={lr:<7g} {status}   final train loss {h['train
        # save progress so a crash / Colab disconnect doesn't lose the sweep
        json.dump({f"{k[0]}|{k[1]}": v for k, v in results.items()}, open(f"
```

```
SGD                        lr=0.003   val acc 0.4778   final train loss 1.51
4   (29s)
SGD                        lr=0.03    val acc 0.5562   final train loss 0.98
1   (23s)
SGD                        lr=0.3     val acc 0.6028   final train loss 0.67
9   (23s)
SGD + momentum (β=0.9)     lr=0.001   val acc 0.5734   final train loss 1.19
6   (23s)
SGD + momentum (β=0.9)     lr=0.01    val acc 0.6954   final train loss 0.78
9   (23s)
SGD + momentum (β=0.9)     lr=0.1     val acc 0.7246   final train loss 0.56
1   (23s)
Adam                       lr=0.0001  val acc 0.6118   final train loss 1.05
4   (24s)
Adam                       lr=0.001   val acc 0.7000   final train loss 0.72
4   (24s)
Adam                       lr=0.01    val acc 0.7404   final train loss 0.54
0   (24s)
```

In [25]:

```
# ---- table of all 9 runs ----
rows = []
for (name, lr), h in results.items():
    rows.append({"optimizer": name, "lr": lr, "final val acc": h["val_acc"][
                 "final train loss": h["train_loss"][-1], "diverged": h["div
df = pd.DataFrame(rows)
pd.set_option("display.float_format", lambda v: f"{v:.4g}")
display(df)

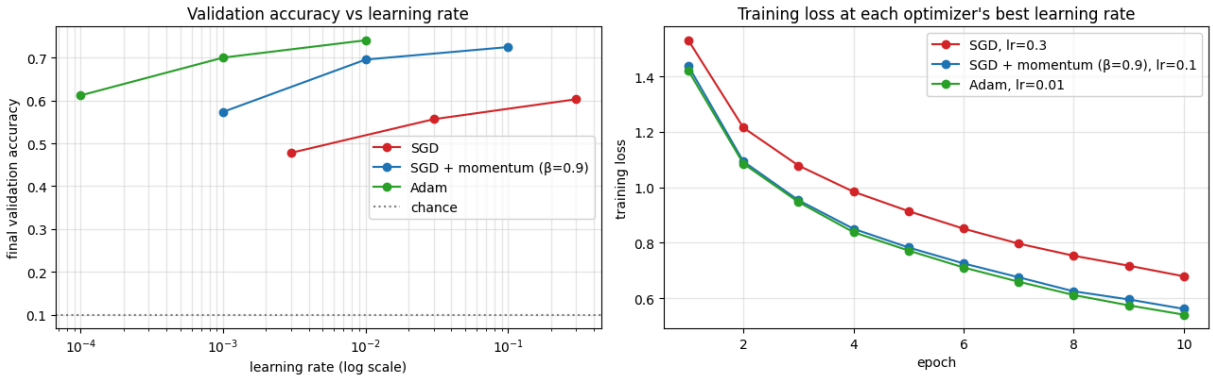
best = {name: max(lrs_, key=lambda lr: results[(name, lr)]["val_acc"][-1])
        for name, (_, lrs_) in GRID.items()}
print("best lr per optimizer:", best)

fig, ax = plt.subplots(1, 2, figsize=(13, 4.2))
# (1) validation accuracy vs learning rate
for (name, (_, lrs_)), c in zip(GRID.items(), ["tab:red", "tab:blue", "tab:g
    ax[0].plot(lrs_, [results[(name, lr)]["val_acc"][-1] for lr in lrs_], "c
ax[0].set_xscale("log"); ax[0].axhline(0.10, color="gray", ls=":", label="ch
ax[0].set(xlabel="learning rate (log scale)", ylabel="final validation accur
ax[0].grid(alpha=0.3, which="both"); ax[0].legend()

# (2) training loss at the best lr of each optimizer
for (name, _), c in zip(GRID.items(), ["tab:red", "tab:blue", "tab:green"]):
    h = results[(name, best[name])]
    ax[1].plot(range(1, len(h["train_loss"]) + 1), h["train_loss"], "o-", cc
ax[1].set(xlabel="epoch", ylabel="training loss", title="Training loss at ea
ax[1].grid(alpha=0.3); ax[1].legend()
plt.tight_layout(); plt.show()
```

	optimizer	lr	final val acc	final train loss	diverged
0	SGD	0.003	0.4778	1.514	False
1	SGD	0.03	0.5562	0.9812	False
2	SGD	0.3	0.6028	0.6786	False
3	SGD + momentum (β=0.9)	0.001	0.5734	1.196	False
4	SGD + momentum (β=0.9)	0.01	0.6954	0.7885	False
5	SGD + momentum (β=0.9)	0.1	0.7246	0.5609	False
6	Adam	0.0001	0.6118	1.054	False
7	Adam	0.001	0.7	0.7237	False
8	Adam	0.01	0.7404	0.5401	False

```
best lr per optimizer: {'SGD': 0.3, 'SGD + momentum (β=0.9)': 0.1, 'Adam': 0.01}
```



Interpretation (2c). The saved table reports best final validation accuracies of **74.04% for Adam** (lr 0.01), **72.46% for momentum** (lr 0.1), and **60.28% for SGD** (lr 0.3). Corresponding training losses are 0.5401, 0.5609 and 0.6786.

All best rates are at the upper edge of their grids, so the optimum is not bracketed. Reducing the rate by 10× lowers accuracy by 4.04 points for Adam, 2.92 for momentum and 4.66 for SGD. Reducing it by 100× lowers accuracy by 12.86, 15.12 and 12.50 points, respectively. These results show substantial sensitivity to learning rate within the tested budget. None of the saved runs diverged; lower rates appear to under-train.

Momentum accumulates consistent gradients and Adam rescales updates per coordinate. Both outperform plain SGD under this protocol, but each setting has only one seed and its own rate grid. The 1.58-point

gap between Adam and momentum is not enough to establish a general ranking.

Task 2 summary

- `MySGD` and `MyAdam` match PyTorch (0 and ≈3e-15).
- On the quadratic, momentum and Adam far outperform SGD.
- On the CNN: Adam 74.04% > momentum 72.46% > SGD 60.28%, with all best rates at the grid edge.

Task 3: Regularization & Residual Connections

3(a) Regularization on 5k training images

Setup: the first 5,000 images of the training split; validation is the same 5k split as before. The model is the batch-norm CNN from Task 2 (≈95k parameters). All runs use SGD with momentum 0.9, lr 0.05, cosine decay, batch 128, 60 epochs, and 3 seeds (mean ± std). "Train accuracy" is measured in eval mode on the clean 5k training set.

Method	Setting
Weight decay	5e-4 (L2)
Dropout	p = 0.5 before the classifier
Flip + crop	random horizontal flip, random 32×32 crop from 4-pixel zero padding
Cutout	one 16×16 zeroed square per image (implemented by hand)
Mixup	$\lambda \sim \mathrm{Beta}(0.4, 0.4)$, mix images and loss (implemented by hand)
All combined	all of the above

In [26]:
```
X5, y5 = Xtr[:5000].to(device), ytr[:5000].to(device)
Xv,  yv  = Xva.to(device), yva.to(device)
print("class counts in the 5k subset:", torch.bincount(y5.cpu()).tolist())

def random_flip(x):
    m = torch.rand(x.size(0), device=x.device) < 0.5
    return torch.where(m[:, None, None, None], x.flip(3), x)
```

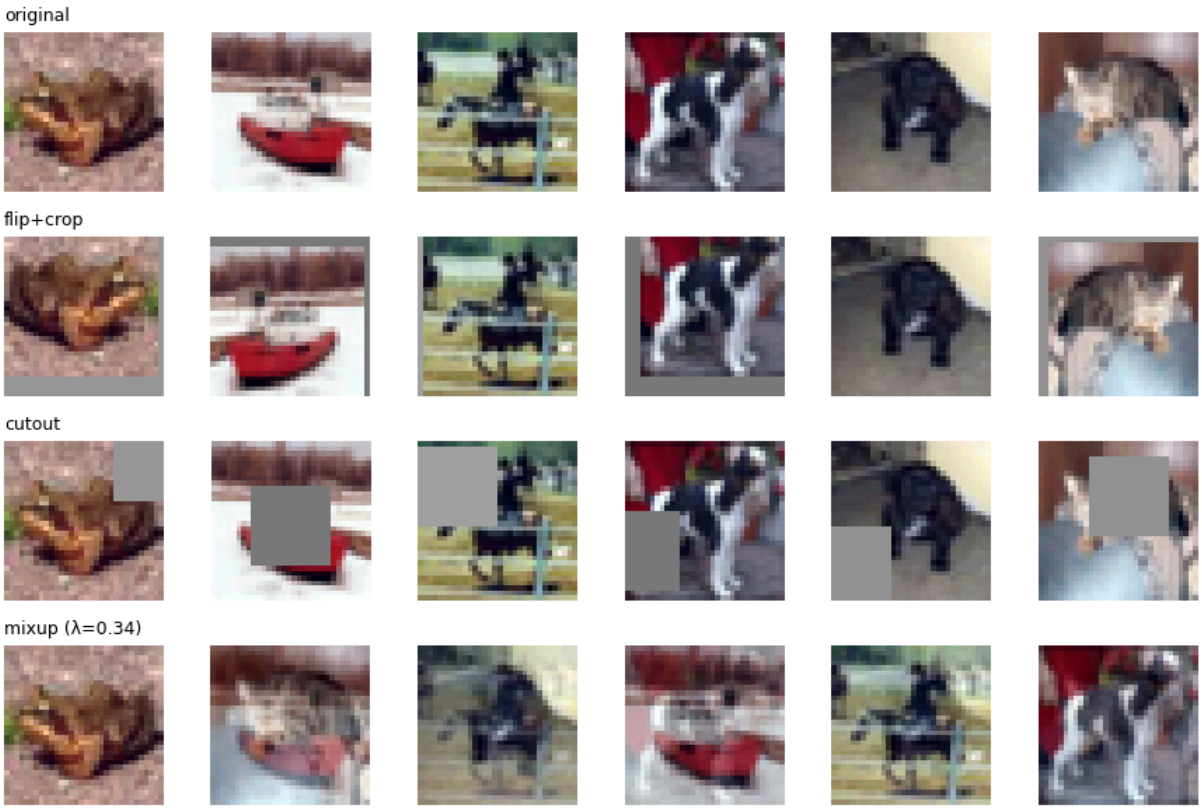
```
def random_crop(x, pad=4):
    N, C, H, W = x.shape
    xp = F.pad(x, (pad, pad, pad, pad))                        # zeros = data
    i = torch.randint(0, 2 * pad + 1, (N,), device=x.device)
    j = torch.randint(0, 2 * pad + 1, (N,), device=x.device)
    rows = i[:, None] + torch.arange(H, device=x.device)       # (N,H)
    cols = j[:, None] + torch.arange(W, device=x.device)       # (N,W)
    n = torch.arange(N, device=x.device)[:, None, None, None]
    c = torch.arange(C, device=x.device)[None, :, None, None]
    return xp[n, c, rows[:, None, :, None], cols[:, None, None, :]]

def cutout(x, size=16):
    N, C, H, W = x.shape
    cy = torch.randint(0, H, (N, 1, 1), device=x.device)
    cx = torch.randint(0, W, (N, 1, 1), device=x.device)
    ys = torch.arange(H, device=x.device)[None, :, None]
    xs = torch.arange(W, device=x.device)[None, None, :]
    m = (ys >= cy - size // 2) & (ys < cy + size // 2) & (xs >= cx - size //
    return x * (~m)[:, None]                                   # patch is cli

def mixup(x, y, alpha=0.4):
    lam = float(np.random.beta(alpha, alpha))
    idx = torch.randperm(x.size(0), device=x.device)
    return lam * x + (1 - lam) * x[idx], y, y[idx], lam

# ---- visual check of the augmentations ----
x0 = X5[:6]
mx, _, _, lam = mixup(x0, y5[:6], alpha=0.4)
rows_ = {"original": x0, "flip+crop": random_crop(random_flip(x0)), "cutout"
fig, ax = plt.subplots(len(rows_), 6, figsize=(9, 6))
for r, (name, imgs) in enumerate(rows_.items()):
    for c in range(6):
        im = imgs[c].permute(1, 2, 0).cpu()
        ax[r, c].imshow((im - im.min()) / (im.max() - im.min())); ax[r, c].a
    ax[r, 0].set_title(name, fontsize=9, loc="left")
plt.tight_layout(); plt.show()
```

```
class counts in the 5k subset: [511, 510, 506, 509, 502, 472, 500, 470, 500,
520]
```



In [27]:

```
EPOCHS_A, SEEDS = 60, [0, 1, 2]

def make_small_cnn(p_drop=0.0):
    return nn.Sequential(
        nn.Conv2d(3, 32, 3, padding=1),   nn.BatchNorm2d(32),  nn.ReLU(), nr
        nn.Conv2d(32, 64, 3, padding=1),  nn.BatchNorm2d(64),  nn.ReLU(), nr
        nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
        nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(p_drop), nn.Linear

@torch.no_grad()
def evaluate(model, X, y, bs=1000):
    model.eval()
    loss, correct = 0.0, 0
    for i in range(0, len(X), bs):
        out = model(X[i:i + bs])
        loss += F.cross_entropy(out, y[i:i + bs], reduction="sum").item()
        correct += (out.argmax(1) == y[i:i + bs]).sum().item()
    return loss / len(X), correct / len(X)

CONFIGS = {
    "Baseline":     dict(),
    "Weight decay": dict(wd=5e-4),
    "Dropout":      dict(p=0.5),
    "Flip+crop":    dict(flipcrop=True),
    "Cutout":       dict(cutout=True),
    "Mixup":        dict(mixup=True),
    "All combined": dict(wd=5e-4, p=0.5, flipcrop=True, cutout=True, mixup=T
}

def run_cfg(cfg, seed, epochs=EPOCHS_A, lr=0.05, bs=128):
```

```
    seed_everything(seed)
    model = make_small_cnn(cfg.get("p", 0.0))
    opt = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9, weight_de
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)
    h = {k: [] for k in ["train_loss", "train_acc", "val_loss", "val_acc"]}
    for ep in range(epochs):
        model.train()
        perm = torch.randperm(len(X5), device=device)
        for i in range(0, len(perm), bs):
            idx = perm[i:i + bs]; x, y = X5[idx], y5[idx]
            if cfg.get("flipcrop"): x = random_crop(random_flip(x))
            if cfg.get("cutout"):   x = cutout(x)
            opt.zero_grad(set_to_none=True)
            if cfg.get("mixup"):
                x, ya, yb, lam = mixup(x, y)
                out = model(x)
                loss = lam * F.cross_entropy(out, ya) + (1 - lam) * F.cross_
            else:
                loss = F.cross_entropy(model(x), y)
            loss.backward(); opt.step()
        sched.step()
        tl, ta = evaluate(model, X5, y5); vl, va = evaluate(model, Xv, yv)
        for k, v in zip(h, [tl, ta, vl, va]): h[k].append(v)
    return h

res_a = {}
for name, cfg in CONFIGS.items():
    t0 = time.time()
    res_a[name] = [run_cfg(cfg, s) for s in SEEDS]
    va = [h["val_acc"][-1] for h in res_a[name]]
    print(f"{name:13s} val acc {np.mean(va):.4f} ± {np.std(va):.4f}   ({time
json.dump(res_a, open("task3a.json", "w"))
```

```
Baseline      val acc 0.6613 ± 0.0020   (65s)
Weight decay  val acc 0.6576 ± 0.0033   (65s)
Dropout       val acc 0.6395 ± 0.0073   (65s)
Flip+crop     val acc 0.6725 ± 0.0010   (67s)
Cutout        val acc 0.6613 ± 0.0018   (66s)
Mixup         val acc 0.6699 ± 0.0005   (66s)
All combined  val acc 0.5908 ± 0.0023   (68s)
```

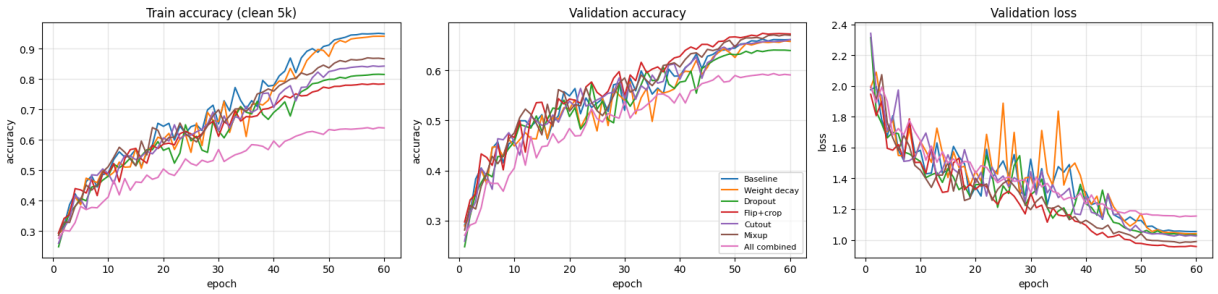
In [28]:

```
rows = []
for name, hs in res_a.items():
    tr = np.array([h["train_acc"][-1] for h in hs]) * 100
    va = np.array([h["val_acc"][-1] for h in hs]) * 100
    gap = tr - va
    rows.append({"method": name,
                 "train acc (%)": f"{tr.mean():.1f} ± {tr.std():.1f}",
                 "val acc (%)":   f"{va.mean():.1f} ± {va.std():.1f}",
                 "gap (pts)":     f"{gap.mean():.1f} ± {gap.std():.1f}"})
display(pd.DataFrame(rows))

def mean_curve(name, key): return np.mean([h[key] for h in res_a[name]], axi
fig, ax = plt.subplots(1, 3, figsize=(17, 4.2))
for name in res_a:
    ep = np.arange(1, EPOCHS_A + 1)
```

```
    ax[0].plot(ep, mean_curve(name, "train_acc"), label=name)
    ax[1].plot(ep, mean_curve(name, "val_acc"), label=name)
    ax[2].plot(ep, mean_curve(name, "val_loss"), label=name)
for a, t, yl in zip(ax, ["Train accuracy (clean 5k)", "Validation accuracy",
    a.set(title=t, xlabel="epoch", ylabel=yl); a.grid(alpha=0.3)
ax[1].legend(fontsize=8)
plt.tight_layout(); plt.show()
```

	method	train acc (%)	val acc (%)	gap (pts)
0	Baseline	94.9 ± 0.4	66.1 ± 0.2	28.8 ± 0.2
1	Weight decay	94.1 ± 0.3	65.8 ± 0.3	28.3 ± 0.5
2	Dropout	81.5 ± 0.8	64.0 ± 0.7	17.6 ± 1.0
3	Flip+crop	78.4 ± 0.3	67.3 ± 0.1	11.1 ± 0.3
4	Cutout	84.3 ± 0.1	66.1 ± 0.2	18.1 ± 0.2
5	Mixup	86.7 ± 0.2	67.0 ± 0.1	19.7 ± 0.1
6	All combined	63.9 ± 0.1	59.1 ± 0.2	4.8 ± 0.3



Interpretation (3a). The baseline overfits: clean training accuracy is 94.9% versus 66.1% validation accuracy, a 28.8-point gap (mean over three seeds).

Flip+crop gives the highest mean validation accuracy (67.3%) and reduces the gap to 11.1 points. Mixup reaches 67.0% with a 19.7-point gap. Their small difference should be treated cautiously with only three seeds and one validation split; the rounded reported standard deviations do not support the earlier claim that they differ by less than one standard deviation.

Cutout matches baseline validation accuracy (66.1%) while reducing clean training accuracy to 84.3%. Weight decay gives 65.8%, a small decrease relative to baseline. Dropout reduces both training accuracy (81.5%) and validation accuracy (64.0%): a smaller gap alone does not establish better generalization.

All combined gives the smallest gap (4.8 points) but the lowest validation accuracy (59.1%), with only 63.9% clean training accuracy. This is consistent with underfitting under the chosen strengths and

60-epoch budget. Combining all methods does not guarantee an improvement.

The curves should be read alongside the clean-set accuracy table: optimization uses augmented or mixed inputs for some methods, while the reported training metrics consistently use unaugmented images in evaluation mode. Results are limited to one 5k training subset, three seeds and hand-picked regularization strengths.

3(b) Plain vs residual networks (depth 6n+2)

CIFAR-style nets: 3×3 conv (16) → three stages of n blocks with 16 / 32 / 64 channels (stride 2 at the start of stages 2 and 3) → global average pooling → linear. Depth $= 6n + 2$ with $n = 1, 3, 9$ gives 8, 20 and 56. Batch norm after every conv, Kaiming-normal init.

The only difference is the shortcut. Residual block: $\mathrm{ReLU}(F(x) + x)$. Plain block: $\mathrm{ReLU}(F(x))$. When the shape changes, the shortcut subsamples by 2 and zero-pads the channels (parameter-free), so a plain and a residual net of the same depth have **identical parameter counts**.

Training: SGD with momentum 0.9, constant lr 0.05, weight decay 5e-4, batch 128, 30 epochs, 45k training images, no augmentation (so training loss reflects optimization). One seed per model.

In [29]:
```
class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride, residual):
        super().__init__()
        self.residual, self.stride, self.pad = residual, stride, (cout - cir
        self.c1 = nn.Conv2d(cin, cout, 3, stride, 1, bias=False); self.b1 =
        self.c2 = nn.Conv2d(cout, cout, 3, 1, 1, bias=False);     self.b2 =

    def shortcut(self, x):
        if self.stride == 1 and self.pad == 0:
            return x
        return F.pad(x[:, :, ::self.stride, ::self.stride], (0, 0, 0, 0, sel

    def forward(self, x):
        out = F.relu(self.b1(self.c1(x)))
        out = self.b2(self.c2(out))
        if self.residual:
            out = out + self.shortcut(x)          # the ONLY difference betw
        return F.relu(out)

class CifarNet(nn.Module):
    def __init__(self, n, residual):
```

```
        super().__init__()
        self.conv = nn.Conv2d(3, 16, 3, 1, 1, bias=False); self.bn = nn.Batc
        blocks, cin = [], 16
        for cout, stride in [(16, 1), (32, 2), (64, 2)]:
            for k in range(n):
                blocks.append(BasicBlock(cin, cout, stride if k == 0 else 1,
        self.blocks = nn.Sequential(*blocks)
        self.fc = nn.Linear(64, 10)
        for m in self.modules():
            if isinstance(m, (nn.Conv2d, nn.Linear)):
                nn.init.kaiming_normal_(m.weight, nonlinearity="relu")
            if isinstance(m, nn.Linear):
                nn.init.zeros_(m.bias)

    def forward(self, x):
        x = F.relu(self.bn(self.conv(x)))
        x = self.blocks(x)
        return self.fc(F.adaptive_avg_pool2d(x, 1).flatten(1))

# sanity: depth = 6n+2 and equal parameter counts
for n in [1, 3, 9]:
    for res in [False, True]:
        m = CifarNet(n, res)
        layers = sum(isinstance(k, (nn.Conv2d, nn.Linear)) for k in m.module
        print(f"n={n} residual={res!s:5}  weight layers={layers:2d} (6n+2={6
```

```
n=1 residual=False  weight layers= 8 (6n+2=8)  params=75,290
n=1 residual=True   weight layers= 8 (6n+2=8)  params=75,290
n=3 residual=False  weight layers=20 (6n+2=20)  params=269,722
n=3 residual=True   weight layers=20 (6n+2=20)  params=269,722
n=9 residual=False  weight layers=56 (6n+2=56)  params=853,018
n=9 residual=True   weight layers=56 (6n+2=56)  params=853,018
```

In [30]:

```
EPOCHS_B, DEPTHS = 30, {8: 1, 20: 3, 56: 9}
XT, yT = Xtr.to(device), ytr.to(device)

def train_net(n, residual, epochs=EPOCHS_B, lr=0.05, bs=128, seed=0):
    seed_everything(seed)
    model = CifarNet(n, residual).to(device)
    opt = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9, weight_de
    h = {"train_loss": [], "val_acc": [], "diverged": False}
    for ep in range(epochs):
        model.train()
        perm = torch.randperm(len(XT), device=device)
        tot, cnt = 0.0, 0
        for i in range(0, len(perm), bs):
            idx = perm[i:i + bs]
            opt.zero_grad(set_to_none=True)
            loss = F.cross_entropy(model(XT[idx]), yT[idx])
            if not torch.isfinite(loss):
                h["diverged"] = True; break
            loss.backward(); opt.step()
            tot += loss.item() * len(idx); cnt += len(idx)
        if h["diverged"]: break
        h["train_loss"].append(tot / cnt)
        h["val_acc"].append(evaluate(model, Xv, yv)[1])
```

```
    h["params"] = sum(p.numel() for p in model.parameters())
    return h

# time one epoch of the deepest model first
t0 = time.time(); _ = train_net(9, True, epochs=1); t1 = time.time() - t0
print(f"ResNet-56: ≈{t1:.0f}s/epoch → whole experiment ≈ {t1 * EPOCHS_B * 3.
```

```
ResNet-56: ≈35s/epoch → whole experiment ≈ 56 min
```

In [31]:

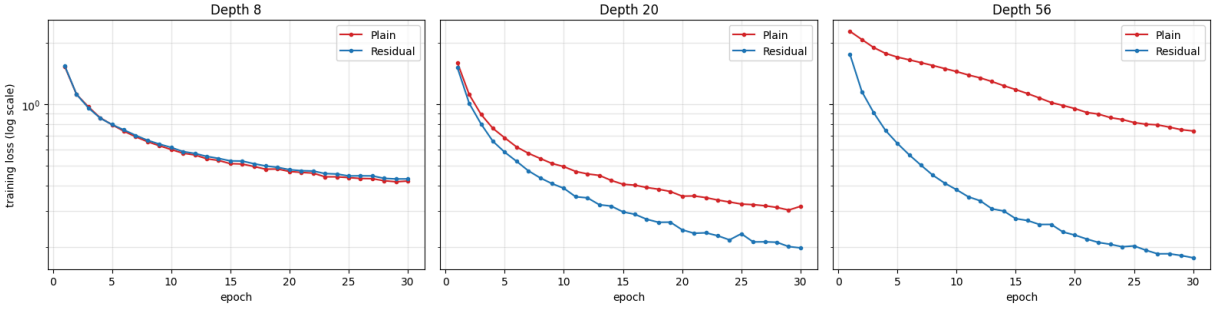
```
res_b = {}
for depth, n in DEPTHS.items():
    for residual in [False, True]:
        name = f"{'Residual' if residual else 'Plain'}-{depth}"
        t0 = time.time()
        res_b[name] = train_net(n, residual)
        h = res_b[name]
        st = "DIVERGED" if h["diverged"] else f"train loss {h['train_loss'][
        print(f"{name:12s} {st}   ({time.time()-t0:.0f}s)")
        json.dump(res_b, open("task3b.json", "w"))
```

```
Plain-8      train loss 0.421  val acc 0.7516   (123s)
Residual-8   train loss 0.431  val acc 0.7244   (131s)
Plain-20     train loss 0.317  val acc 0.7708   (344s)
Residual-20  train loss 0.199  val acc 0.7758   (360s)
Plain-56     train loss 0.739  val acc 0.6290   (1007s)
Residual-56  train loss 0.178  val acc 0.7614   (1051s)
```

In [32]:

```
fig, ax = plt.subplots(1, 3, figsize=(16, 4.2), sharey=True)
for a, depth in zip(ax, DEPTHS):
    for fam, c in [("Plain", "tab:red"), ("Residual", "tab:blue")]:
        h = res_b[f"{fam}-{depth}"]
        a.plot(range(1, len(h["train_loss"]) + 1), h["train_loss"], "o-", ms
    a.set(title=f"Depth {depth}", xlabel="epoch"); a.set_yscale("log"); a.gr
ax[0].set_ylabel("training loss (log scale)")
plt.tight_layout(); plt.show()

rows = []
for depth in DEPTHS:
    for fam in ["Plain", "Residual"]:
        h = res_b[f"{fam}-{depth}"]
        rows.append({"depth": depth, "family": fam, "params": f"{h['params']
                     "final train loss": h["train_loss"][-1], "final val acc
                     "diverged": h["diverged"]})
pd.set_option("display.float_format", lambda v: f"{v:.3f}")
display(pd.DataFrame(rows))
```



	depth	family	params	final train loss	final val acc (%)	diverged
0	8	Plain	75,290	0.421	75.160	False
1	8	Residual	75,290	0.431	72.440	False
2	20	Plain	269,722	0.317	77.080	False
3	20	Residual	269,722	0.199	77.580	False
4	56	Plain	853,018	0.739	62.900	False
5	56	Residual	853,018	0.178	76.140	False

Interpretation (3b). Plain and residual networks at each depth have identical parameter counts; the shortcut is the only architectural difference.

Plain-8, Plain-20 and Plain-56 finish with training losses **0.421, 0.317 and 0.739**. The deeper plain network is harder to optimize: Plain-56 has a higher training loss than Plain-20, which cannot be explained solely by overfitting.

Residual-8, Residual-20 and Residual-56 finish at **0.431, 0.199 and 0.178**. Shortcuts provide a direct gradient path through $F(x) + x$ and let a block approach an identity mapping when $F(x)$ is small. The benefit is largest at depth 56 (0.178 versus 0.739 training loss).

Validation accuracy does not strictly track training loss. At depth 8, plain achieves 75.16% versus residual 72.44%; at depth 20, residual achieves 77.58% versus plain 77.08%; at depth 56, residual achieves 76.14% versus plain 62.90%. Residual-20 has the highest observed validation accuracy. Residual-56 fits training better without improving validation accuracy over Residual-20, consistent with a generalization limitation under this recipe.

These are single-seed, final-epoch comparisons with a constant learning rate and no augmentation. Small accuracy differences do not establish a reliable ranking, and longer or tuned training might change the result.

Task 3 summary

- **3(a):** baseline gap 28.8 points; flip+crop gives 67.3% validation accuracy and mixup 67.0%. All combined underfits (59.1%).
- **3(b):** Plain-56 has higher training loss than Plain-20 (0.739 versus 0.317), while residual training loss improves from 0.199 to 0.178.

- Residual-20 has the highest observed validation accuracy (77.58%); the shortcut advantage is strongest at depth 56.

Task 4: Best CNN (≤ 500k parameters, ≤ 20 epochs)

Design: a CIFAR-style ResNet, 3 stages × 3 blocks (depth 20) with widths 20 / 40 / 80 (≈420k parameters), batch norm, Kaiming init, global average pooling, identity shortcuts with parameter-free zero-padding.

Training recipe: SGD with Nesterov momentum 0.9, one-cycle lr schedule, weight decay, label smoothing 0.1, flip + crop + cutout (8×8), batch 128, 20 epochs, 45k training images.

Protocol: the learning rate and weight decay are tuned on the 5k validation split. The ablation removes one design choice at a time from the tuned model. All numbers are final-epoch validation accuracy (one seed). The test set is evaluated **once**, for the final model only.

In [33]:
```
import json, math, pandas as pd   # needs Task 3 cells: BasicBlock, random_f

class Net(nn.Module):
    def __init__(self, n=3, widths=(20, 40, 80), residual=True):
        super().__init__()
        self.conv = nn.Conv2d(3, widths[0], 3, 1, 1, bias=False); self.bn =
        blocks, cin = [], widths[0]
        for cout, stride in zip(widths, (1, 2, 2)):
            for k in range(n):
                blocks.append(BasicBlock(cin, cout, stride if k == 0 else 1,
        self.blocks = nn.Sequential(*blocks)
        self.fc = nn.Linear(widths[-1], 10)
        for m in self.modules():
            if isinstance(m, (nn.Conv2d, nn.Linear)): nn.init.kaiming_normal
            if isinstance(m, nn.Linear): nn.init.zeros_(m.bias)

    def forward(self, x):
        x = F.relu(self.bn(self.conv(x)))
        return self.fc(F.adaptive_avg_pool2d(self.blocks(x), 1).flatten(1))

for res in [True, False]:
    p = sum(q.numel() for q in Net(residual=res).parameters())
    print(f"residual={res!s:5}  params={p:,}  (limit 500,000)")
assert sum(q.numel() for q in Net().parameters()) <= 500_000
```

```
residual=True   params=420,670  (limit 500,000)
residual=False  params=420,670  (limit 500,000)
```

In [34]:
```
BASE = dict(residual=True, aug="flipcrop+cutout", wd=5e-4, opt="sgd", lr=0.1
EPOCHS_4, BS_4 = 20, 128

def train_cfg(cfg, epochs=EPOCHS_4, bs=BS_4, seed=0):
    seed_everything(seed)
    model = Net(residual=cfg["residual"]).to(device)
    if cfg["opt"] == "sgd":
        lr = cfg["lr"]; opt = torch.optim.SGD(model.parameters(), lr=lr, mom
    else:
        lr = cfg.get("lr_adam", 2e-3); opt = torch.optim.Adam(model.paramete
    steps = math.ceil(len(Xtr) / bs)
    sched = (torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=lr, epochs=epoc
             if cfg["sched"] == "onecycle" else None)
    h = {"train_loss": [], "val_loss": [], "val_acc": [], "diverged": False}
    for ep in range(epochs):
        model.train()
        perm = torch.randperm(len(Xtr), device=device); tot, cnt = 0.0, 0
        for i in range(0, len(perm), bs):
            idx = perm[i:i + bs]; x, y = Xtr[idx], ytr[idx]
            if "flipcrop" in cfg["aug"]: x = random_crop(random_flip(x))
            if "cutout" in cfg["aug"]:   x = cutout(x, 8)
            opt.zero_grad(set_to_none=True)
            loss = F.cross_entropy(model(x), y, label_smoothing=cfg["ls"])
            if not torch.isfinite(loss): h["diverged"] = True; break
            loss.backward(); opt.step()
            if sched: sched.step()
            tot += loss.item() * len(idx); cnt += len(idx)
        if h["diverged"]: break
        vl, va = evaluate(model, Xva, yva)
        h["train_loss"].append(tot / cnt); h["val_loss"].append(vl); h["val_
    return model, h

t0 = time.time(); _ = train_cfg(BASE, epochs=1); t1 = time.time() - t0
print(f"≈{t1:.0f}s/epoch → one 20-epoch run ≈ {t1 * 20 / 60:.1f} min; tuning
```

```
≈22s/epoch → one 20-epoch run ≈ 7.4 min; tuning (4 runs) + ablation (6 runs)
≈ 74 min
```

Tuning on the validation split

First the learning rate (0.05 / 0.1 / 0.2, weight decay 5e-4), then weight decay (5e-4 vs 1e-3) at the best learning rate. The test set is not used.

In [35]:
```
tune, tune_models = {}, {}
def run_tune(tag, cfg):
    t0 = time.time(); m, h = train_cfg(cfg)
    tune[tag], tune_models[tag] = h, m
    st = "DIVERGED" if h["diverged"] else f"val acc {h['val_acc'][-1]:.4f}"
    print(f"{tag:22s} {st}   ({time.time()-t0:.0f}s)")
    json.dump({"tune": tune}, open("task4_tune.json", "w"))

for lr in [0.05, 0.1, 0.2]:
```

```
    run_tune(f"lr={lr}, wd=5e-4", {**BASE, "lr": lr})
acc = lambda t: tune[t]["val_acc"][-1] if not tune[t]["diverged"] else 0
best_lr = max([0.05, 0.1, 0.2], key=lambda lr: acc(f"lr={lr}, wd=5e-4"))
run_tune(f"lr={best_lr}, wd=1e-3", {**BASE, "lr": best_lr, "wd": 1e-3})

best_tag = max(tune, key=acc)
FINAL = {**BASE, "lr": float(best_tag.split(",")[0][3:]), "wd": float(best_t
print("\nFINAL config:", FINAL)
display(pd.DataFrame([{"setting": t, "final val acc (%)": acc(t) * 100} for
```

```
lr=0.05, wd=5e-4       val acc 0.8830   (442s)
lr=0.1, wd=5e-4        val acc 0.9016   (441s)
lr=0.2, wd=5e-4        val acc 0.9070   (441s)
lr=0.2, wd=1e-3        val acc 0.9008   (441s)

FINAL config: {'residual': True, 'aug': 'flipcrop+cutout', 'wd': 0.0005, 'op
t': 'sgd', 'lr': 0.2, 'ls': 0.1, 'sched': 'onecycle'}
```

	setting	final val acc (%)
0	lr=0.05, wd=5e-4	88.300
1	lr=0.1, wd=5e-4	90.160
2	lr=0.2, wd=5e-4	90.700
3	lr=0.2, wd=1e-3	90.080

Tuning notes. Validation accuracy rises with the learning rate: 88.3% (0.05), 90.2% (0.1), 90.7% (0.2). The best rate is at the edge of the grid, so a higher rate was not tested. The 0.1 → 0.2 gain (0.5 pts) is about the size of the noise: with 5k validation images the standard error of an accuracy near 90% is about 0.4 pts, and each setting was run once. Weight decay 1e−3 at lr 0.2 gave 90.1%, which is 0.6 pts below 5e−4, so I kept **lr = 0.2, wd = 5e−4**. Tuning was small (4 runs) and used only the validation split.

Ablation

Each row removes **one** design choice from the tuned final model:

Row	Change
No shortcut	plain network (same parameters)
No augmentation	no flip, crop or cutout
No weight decay	`wd = 0`
Adam	Adam, lr 2e−3 (untuned), instead of SGD
No label smoothing	`ls = 0`

Row	Change
Constant lr	no one-cycle schedule (lr fixed at the tuned peak value)

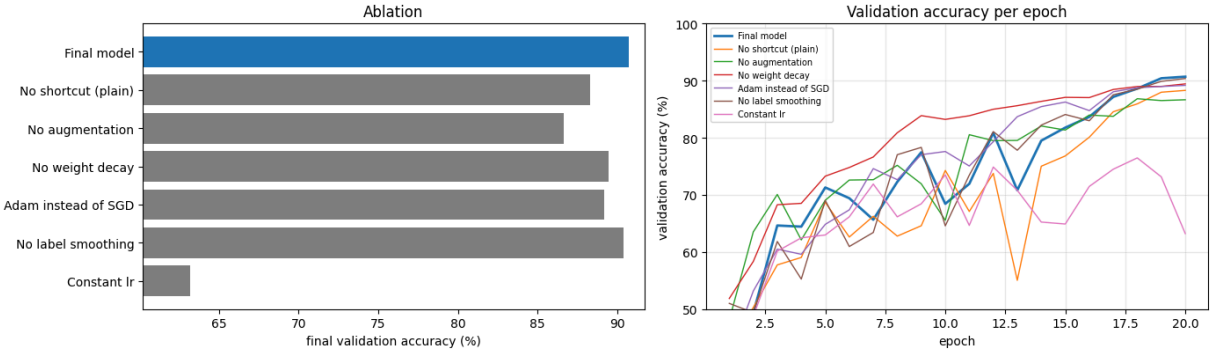
In [36]:
```
ABL = {
    "Final model":           {},
    "No shortcut (plain)":   dict(residual=False),
    "No augmentation":       dict(aug="none"),
    "No weight decay":       dict(wd=0.0),
    "Adam instead of SGD":   dict(opt="adam"),
    "No label smoothing":    dict(ls=0.0),
    "Constant lr":           dict(sched="const"),
}
abl = {"Final model": tune[best_tag]}                    # reuse the tuned
for name, delta in ABL.items():
    if name == "Final model": continue
    t0 = time.time(); m, h = train_cfg({**FINAL, **delta}); del m
    abl[name] = h
    st = "DIVERGED" if h["diverged"] else f"val acc {h['val_acc'][-1]:.4f}"
    print(f"{name:22s} {st}   ({time.time()-t0:.0f}s)")
    json.dump({"tune": tune, "abl": abl}, open("task4_abl.json", "w"))
```

```
No shortcut (plain)    val acc 0.8830   (430s)
No augmentation        val acc 0.8666   (437s)
No weight decay        val acc 0.8944   (440s)
Adam instead of SGD    val acc 0.8918   (441s)
No label smoothing     val acc 0.9038   (439s)
Constant lr            val acc 0.6322   (437s)
```

In [37]:
```
base_acc = abl["Final model"]["val_acc"][-1] * 100
rows = [{"variant": n, "final val acc (%)": (h["val_acc"][-1] * 100 if not h
         "Δ vs final (pts)": (h["val_acc"][-1] * 100 - base_acc if not h["di
         "final train loss": h["train_loss"][-1] if h["train_loss"] else flo
pd.set_option("display.float_format", lambda v: f"{v:.2f}")
display(pd.DataFrame(rows))

fig, ax = plt.subplots(1, 2, figsize=(14, 4.2))
names = list(abl); accs = [abl[n]["val_acc"][-1] * 100 for n in names]
ax[0].barh(names[::-1], accs[::-1], color=["tab:blue" if n == "Final model"
ax[0].set(xlabel="final validation accuracy (%)", xlim=(min(accs) - 3, max(a
for n, h in abl.items():
    ax[1].plot(range(1, len(h["val_acc"]) + 1), np.array(h["val_acc"]) * 100
ax[1].set(xlabel="epoch", ylabel="validation accuracy (%)", ylim=(50, 100),
ax[1].grid(alpha=0.3); ax[1].legend(fontsize=7)
plt.tight_layout(); plt.show()
```

	variant	final val acc (%)	Δ vs final (pts)	final train loss
0	Final model	90.70	0.00	0.70
1	No shortcut (plain)	88.30	−2.40	0.76
2	No augmentation	86.66	−4.04	0.52
3	No weight decay	89.44	−1.26	0.72
4	Adam instead of SGD	89.18	−1.52	0.73
5	No label smoothing	90.38	−0.32	0.25
6	Constant lr	63.22	−27.48	0.99



Interpretation (ablation). Validation accuracy of the final model is 90.7%. Drop when each choice is removed (one seed, 5k validation images, standard error ≈ 0.4 pts):

Removed	Drop (pts)	Interpretation (single seed)
One-cycle lr schedule (constant lr)	27.5	large observed effect
Augmentation	4.0	large observed effect
Shortcut	2.4	needs repeated-seed confirmation
SGD → Adam (untuned lr)	1.5	borderline
Weight decay	1.3	borderline
Label smoothing	0.3	no (within noise)

- **Lr schedule matters most.** A constant lr of 0.2 never settles. The validation curve is erratic and peaks at about 76% around epoch 18 before falling to 63%. Decaying the lr removes this noise and gives the large final jump seen in the other curves.
- **Augmentation** is the second largest. Without it, training loss is lower (0.52 vs 0.70) but validation accuracy is 4 pts worse, which

indicates overfitting. (Losses are not directly comparable, since augmented batches are harder.)

- **Shortcut:** the plain net has a higher training loss (0.76 vs 0.70) and 2.4 pts lower accuracy, consistent with the optimization advantage in 3(b). With this recipe the residual net wins. Task 3(b) also shows a small validation advantage for Residual-20 (77.58% versus 77.08%); the larger difference here may reflect the changed training recipe, though that explanation was not isolated experimentally.
- **Adam and weight decay** each cost 1–1.5 pts. The Adam result is not a fair verdict on Adam, since its lr was not tuned. The no-weight-decay run is ahead mid-training (about 84% vs 77% at epoch 9) but ends lower.
- **Label smoothing** has no clear effect (−0.3 pts). Its training loss (0.25) is not comparable to the others, because the smoothed loss has a higher floor.

Caveats: one seed per variant, one change at a time (interactions are not tested), and differences below about 1 pt are not reliable.

Final test evaluation (once)

The final model is the tuned model from the ablation (trained on the 45k training split only). The test images are normalized with the **training** statistics. This cell runs **once**; no decision is made after seeing it.

In [38]:
```
assert not globals().get("TEST_DONE", False), "Test set already used - do nc
final_model = tune_models[best_tag]
Xte = ((X_test_raw - mean) / std).to(device); yte = y_test.to(device)
test_loss, test_acc = evaluate(final_model, Xte, yte)
TEST_DONE = True
n_par = sum(q.numel() for q in final_model.parameters())
print(f"Final model: {n_par:,} parameters, {EPOCHS_4} epochs")
print(f"Validation accuracy: {abl['Final model']['val_acc'][-1]*100:.2f}%")
print(f"TEST accuracy      : {test_acc*100:.2f}%   (loss {test_loss:.3f})")
```

```
Final model: 420,670 parameters, 20 epochs
Validation accuracy: 90.70%
TEST accuracy      : 90.08%   (loss 0.347)
```

Result. The final model (420,670 parameters, 20 epochs) reaches **90.08% test accuracy**, evaluated once. Validation accuracy was 90.70%. The 0.6 pt difference is about the size of the validation noise (≈0.4 pts), plus a small optimistic bias from choosing the best of 4 tuning runs on the validation split. The test set was used for no decision.

Task 4 summary

- **Model:** ResNet-20-style, widths 20/40/80, 420,670 parameters (limit 500k), 20 epochs.
- **Recipe (tuned on validation):** SGD with Nesterov momentum, one-cycle lr (peak 0.2), wd 5e-4, label smoothing 0.1, flip + crop + cutout.
- **Result:** 90.70% validation, **90.08% test** (evaluated once).
- **Ablation:** the lr schedule (−27.5 pts) and augmentation (−4.0) matter most, followed by the shortcut (−2.4). Adam (untuned), weight decay and label smoothing cost 1.5, 1.3 and 0.3 pts, which are borderline or within noise.